

ota-manager TypeScript Typecheck Triage — EVIDENCE

Revision: 1 **Last modified:** 2026-07-09T00:00:00Z

Scope

Stream P, single-resource-owner for the ota-manager build this stream (§11.4.119). Triageed the 118 pre-existing TypeScript type errors in `clients/ota-manager` (`pnpm tsc --noEmit -p tsconfig.json`), fixed the router-independent subset, and left every error that depends on the operator-gated `react-router-dom` → `@tanstack/react-router` reconciliation untouched. No git commands were run (per instructions — this is the conductor's job). No routing was wired.

Result summary

Metric	Value
Errors before	118
Errors after	85
Net fixed	33
Files fully fixed (0 errors remaining)	13
Files left untouched (bucket a, router/unrouted-page)	11
<code>pnpm build</code> (vite)	PASS , exit 0
<code>pnpm vitest run</code>	PASS , 9 files / 36 tests, exit 0

Captured command output: `typecheck_before.log` (118 errors, full list), `typecheck_after.log` (85 errors, full list), `build.log`, `vitest.log` — all in this directory.

How the reproduction was run

```
cd clients/ota-manager
pnpm install                    # "Already up to date"
pnpm tsc --noEmit -p tsconfig.json # 118 errors (exit 2) -
typecheck_before.log
```

`package.json` has no separate test script; the project's test runner is invoked directly as `pnpm vitest run` (`vitest` is a `devDependency`, `vitest.config.ts` present at the client root). `pnpm build` runs `vite build` per `package.json`'s build script (it does not itself invoke `tsc`, hence build passing does not by itself prove zero type errors — the typecheck above is the authoritative type-error count).

Root-cause finding that unlocked most of the fixable set

`src/lib/api-client.ts` already defines the full wire-type contract for every domain (devices, releases, deployments, groups, audit) matching the real Go server (`server/internal/api/wire.go`, `handlers_delta.go`, `audit_wire.go`, `handlers_audit.go` — read for reference only, never modified). But `src/types/api.ts` — the barrel file every hook imports from — only re-exported a small subset of those types. Every hook file importing a type that existed in `api-client.ts` but was not in that subset failed with `TS2305: has no exported member`. This is the majority of the 118 errors and has **nothing to do with routing** — it is a plain re-export gap. Fixing `types/api.ts` (and, for the one domain — `Artifacts/Deltas` — whose types did not exist anywhere yet, adding them to `api-client.ts` after reading the real server contract) resolved this class mechanically, with no design/router decision involved.

Side effect, disclosed honestly: because those imports previously failed to resolve, TypeScript had been silently widening the affected hook return values to effectively `any`, which was *hiding* several genuine, pre-existing data-shape bugs in downstream consumers (a hook returning a paginated `{items, next_cursor}` wrapper being treated by its page as a flat array). Fixing the root cause makes those hidden bugs visible again as real errors. None of these are new bugs I introduced — they were always broken, just masked. Every one of them lands in a file that is itself an unrouted/off-limits page per bucket (a) below, so they are left untouched for the same reason as the rest of that cluster.

Bucket definitions

- **(a) ROUTER/UNROUTED-PAGE dependent** — off limits. The file is either the router config itself, a page not present in the live route-tree.gen.ts (confirmed: the only routes actually registered and rendered are /login and /dashboard via MainLayout — every other feature page is unrouted), or a live component whose specific error is a react-router-dom-style API used against @tanstack/react-router's different Link/useNavigate contract. Left entirely untouched.
- **(b) Independent genuine type bug** — safe to fix without any router/UX decision. Fixed.

Full categorization table

Bucket (b) — FIXED

File	Original error(s) (line:code)	Reason it's bucket (b)	Fix a
src/__tests__/features/releases.test.tsx	72:24 TS6133, 72:31 TS6133	Unused destructured params in a Vitest mock — no runtime/router meaning	Remo onVal Selec destr
src/components/ui/form.tsx	7:7 TS6133, 12:7 TS6133	FormFieldContext/ FormItemContext are never provided (.Provider) nor consumed anywhere in the file — genuinely dead local declarations	Remo conte
src/components/ui/toast.tsx	4:20 TS6133, 8:7 TS6133	VariantProps and the cva-built ToastActionVariants are unused; nothing in the file's public API (Toast, ToastProvider, useToast) touches them	Remo impo cva c
src/features/auth/auth-guard.tsx	2:10 TS2614	login-page.tsx uses export default; the import used named-import syntax — a plain	Chan { Log impor

File	Original error(s) (line:code)	Reason it's bucket (b)	Fix a
		import-statement bug, unrelated to routing	
src/features/dashboard/dashboard-page.tsx	13:3 TS6133	Unused Server icon import from lucide-react	Remove the icon note <i>different</i> mask surface; l see n
src/hooks/use-artifacts.ts	7:3,8:3,9:3,10:3,11:3 TS2305; 13:27 TS2305; 48:52 TS18046	The 5 imported types + apiMultipartPost genuinely did not exist anywhere in the client. Verified the real server contract (server/internal/api/wire.go ArtifactUploadMetadata/ Artifact, handlers_delta.go DeltaRegister/ DeltaView, handleFindDelta query params base/target) — a mechanical, non-guessed fix, zero router involvement	Add Delta Artifact Delta Delta inter apiMu helpe re-ex api.t (data resol once apiMu prop
src/hooks/use-audit.ts	6:15, 6:29 TS2305	AuditLogList/ AuditFilter did not exist; verified against server/internal/api/audit_wire.go (AuditLogList{items, next_cursor}) and handlers_audit.go query params (action, resource_type, since, until, cursor, limit)	Add to ap expo api.t
src/hooks/use-deployments.ts	7:3,8:3,9:3,11:3,12:3,13:3,14:3,15:3 TS2305/TS2724	All 8 types (Deployment, DeploymentList, CreateRolloutRequest, RolloutState,	Add expo

File	Original error(s) (line:code)	Reason it's bucket (b)	Fix a
src/hooks/use-devices.ts	7:3,10:3,11:3,12:3,13:3,14:3 TS2305	RolloutDecision, RecallRequest, RollbackList, plus DeploymentStatus) already exist fully-defined in api-client.ts — pure re-export gap DeviceRegistration, DeviceListFilter, DeviceList, TelemetryHistory, TelemetryFilter, TelemetryOverview already exist in api-client.ts — re-export gap GroupList, UpdateGroupRequest, AddGroupMembersRequest, AddGroupMembersResult, GroupMembers already exist — re-export gap. Separately: useCreateGroup/ useUpdateGroup read data.group_id (optional field on Group) as a query-cache key needing a required string; verified server truth (server/internal/store/store.go Group{ID, Name, Description, CreatedAt} — no group_id field on the store row at all) confirms id is the real, always-present identifier	Add re-export
src/hooks/use-groups.ts	8:3,10:3,11:3,12:3,13:3 TS2305/ TS2724; 55:49, 70:49 TS2345		Add re-export
	8:15 TS2305		Add re-export

File	Original error(s) (line:code)	Reason it's bucket (b)	Fix a
src/hooks/use-projects.ts		Project already exists in api-client.ts — re-export gap	
src/hooks/use-releases.ts	7:3,8:3,9:3 TS2305 (+ 1 latent bug surfaced after the re-export fix: 59:51 TS2345)	Release, ReleaseList, ReleaseFilter already exist — re-export gap. The surfaced bug: useCreateRelease's onSuccess read data.release_id (optional on ReleaseResponse/Release) as a cache key; id is the required field	Add expo data.
src/lib/permissions.ts	7:15, 7:21 TS2305	Role, ProjectAccess already exist — re-export gap	Add expo

13 files, 0 remaining errors in every one of them.

Bucket (a) — LEFT UNTOUCHED (router/unrouted-page dependent)

Confirmed by reading src/route-tree.gen.ts (the tree actually consumed by src/main.tsx's createRouter({ routeTree })): the **only** routes live in the running app are /login and /dashboard (via MainLayout → Sidebar + Topbar + Outlet). Every other feature page below is not reachable through the app at all today.

File	Error count (before → after)	Why it's bucket (a)
src/routes/index.tsx	10 → 10	This is the router-migration file: hand-written createFileRoute(...) calls for every feature route, none of which are registered in the live route-tree.gen.ts. Every error is TS2345: Argument of type "<path>" is not assignable to parameter of type 'undefined' — a direct consequence of the route not being

File	Error count (before → after)	Why it's bucket (a)
src/features/ deployments/ deployment- detail- page.tsx	12 → 14	registered. This file isn't even imported by <code>main.tsx/App.tsx</code>. Fixing it is wiring the router. Unrouted page. Mixes genuine router-API errors (<code>useNavigate({to: "/deployments"})/useParams({from: "/deployments/\$deploymentId"})</code> against an unregistered route; literal-type errors), incomplete-hook-shape errors (<code>useToast()</code> destructuring <code>{toast}</code> when the real <code>ToastContextValue</code> shape is <code>{toasts, addToast, removeToast}</code>; <code>useEvaluateRollout/useRecall</code> call-shape mismatches), and 2 newly-surfaced (previously any-masked) Property 'id' does not exist on type 'DeploymentStatus' errors (the real field is <code>deployment_id</code>) — same off-limits data/router rewiring cluster.
src/features/ deployments/ deployments- page.tsx	10 → 11	Unrouted page; same <code>useNavigate/route-literal + toast-shape + CreateDeploymentRequest</code> shape-mismatch pattern, plus 1 newly-surfaced <code>DeploymentList → Deployment[]</code> cast error (previously masked).
src/features/ devices/ device-detail- page.tsx	12 → 12	Unrouted page; <code>useParams({from: "/devices/\$deviceId"})</code> against an unregistered route, and the page destructures a bespoke <code>{device, updates, groups, loading, refresh}</code> shape out of a plain <code>UseQueryResult</code> (the real react-query hook contract) — a hook-layer rewrite, not an isolated bug.
src/features/ devices/	8 → 9	Unrouted page; same <code>UseQueryResult</code>-vs-bespoke-shape

File	Error count (before → after)	Why it's bucket (a)
devices- page.tsx		mismatch, plus 1 newly-surfaced search property not in DeviceListFilter (previously masked).
src/features/ groups/groups- page.tsx	4 → 6	Unrouted page; toast-shape + CreateGroupRequest shape mismatch, plus 2 newly-surfaced GroupList-vs-Group[] errors (previously masked by the unresolved import).
src/features/ releases/ releases- page.tsx	13 → 15	Unrouted page; toast-shape + CreateReleaseRequest/ReleaseFilter shape mismatches + Release import via the (also-unrouted) useReleases hook, plus 2 newly-surfaced Artifact/ReleaseList array-shape errors (previously masked).
src/features/ layout/app- layout.tsx	3 → 3	Entirely unwired alternate layout (superseded by main-layout.tsx+sidebar.tsx+topbar.tsx, the ones actually in route-tree.gen.ts; zero importers anywhere in src/). Its errors are useNavigate("/login") called with a bare string (the react-router-dom calling convention) against @tanstack/react-router's object-argument contract, plus useLogout() destructured as a useMutation result when it is actually a plain () => void — both are exactly the react-router-dom → @tanstack/react-router reconciliation this task calls out as off-limits.
src/features/ layout/ sidebar.tsx	2 → 2	Live component (rendered via the active MainLayout), but the specific error is the router-API mismatch itself: <Link className={{(isActive)} => ...}>

File	Error count (before → after)	Why it's bucket (a)
		is react-router-dom's NavLink render-prop pattern; @tanstack/react-router's Link.className is a plain string, not a function. This is the textbook case the task names explicitly — fixing it means choosing the tanstack-router replacement pattern (activeProps, data-status CSS, etc.), an operator/design decision. The original trivial TS6133 (unused Server import) was fixed (bucket b, see above table). Fixing the types/api.ts re-export gap then surfaced a <i>different</i>, previously-masked error: useAuditLog() (via @/hooks/useAuditLog → ./use-audit) now correctly resolves to AuditLogList ({items, next_cursor}), but the page still assigns it directly to an ActivityEvent[]-typed slot. This file is reachable only through the dead src/routes/index.tsx and a test mock — not through the live route tree — so it belongs to the same unrouted/off-limits cluster as the rest, and the fix (unwrap .items, or redesign the activity-feed hook) is exactly the kind of data/router-layer rework this task is scoped to avoid. Same root cause as dashboard-page.tsx immediately above: useAuditLog() now resolves to the real paginated AuditLogList wrapper instead of silently degrading to any, and this page's .filter().length calls assume a flat array. audit-page.tsx is also not present in the live route-
src/features/dashboard/dashboard-page.tsx	1 → 1 (different error)	
src/features/audit/audit-page.tsx	0 → 2 (new)	

File	Error count (before → after)	Why it's bucket (a)
		tree.gen.ts — it is unrouted — so this is the same off-limits cluster, not a new file-outside-the-cluster regression.

11 files, 85 errors total, all confined to the router/unrouted-page cluster.

Honest boundary (§11.4.6 / §11.4.108)

- The types/api.ts re-export fix and the Artifact/Delta/Audit type additions to api-client.ts are grounded in the real Go server contract (server/internal/api/wire.go, handlers_delta.go, audit_wire.go, handlers_audit.go — read only, never modified, per the “do not touch server/” instruction).
- Fixing that root cause is a net improvement (33 fewer errors) but it is **not** free of side effects: it un-masks 4 previously-hidden latent bugs (2 in audit-page.tsx, 1 in dashboard-page.tsx, and the .id vs deployment_id/search/GroupList/Artifact shape mismatches spread across the already off-limits deployment/device/group/release pages). None of these are new defects I introduced — they were always present, masked by TypeScript’s any-fallback on the unresolved imports. All of them land inside files that are independently classified bucket (a) for router/unrouted reasons, so none required touching an off-limits file to achieve the fix, and none change what the operator still needs to decide for the router migration.
- **85 bucket-(a) errors remain, correctly left for the operator-gated react-router-dom → @tanstack/react-router reconciliation** (and the accompanying data-hook-layer rewrite these 7 feature pages need once they are wired into route-tree.gen.ts). No routing was wired. No file under dashboard/, design-systems/, server/, or any token .css was touched. No git command was run.

Files changed (all under `clients/ota-manager/src`)

- `src/lib/api-client.ts` — added `Artifact`, `ArtifactUploadMetadata`, `DeltaRegisterRequest`, `DeltaView`, `DeltaFindParams`, `AuditFilter`, `AuditLogList`, `apiMultipartPost()`.
- `src/types/api.ts` — expanded the re-export barrel to cover every type already defined in `api-client.ts` (was re-exporting a small subset).
- `src/hooks/use-groups.ts` — `data.group_id` → `data.id` (2 call sites).
- `src/hooks/use-releases.ts` — `data.release_id` → `data.id` (1 call site).
- `src/features/auth/auth-guard.tsx` — default-import fix for `LoginPage`.
- `src/features/dashboard/dashboard-page.tsx` — removed unused `Server` icon import.
- `src/components/ui/form.tsx` — removed dead `FormFieldContext`/`FormItemContext`.
- `src/components/ui/toast.tsx` — removed unused `VariantProps` import and dead `ToastActionVariants` const.
- `src/__tests__/features/releases.test.tsx` — removed unused destructured mock params.

Captured command outputs

- `typecheck_before.log` — full 118-error `tsc --noEmit` output.
- `typecheck_after.log` — full 85-error `tsc --noEmit` output (post-fix).
- `build.log` — `pnpm build (vite)`, exit 0.
- `vitest.log` — `pnpm vitest run`, 9 files / 36 tests passed, exit 0.